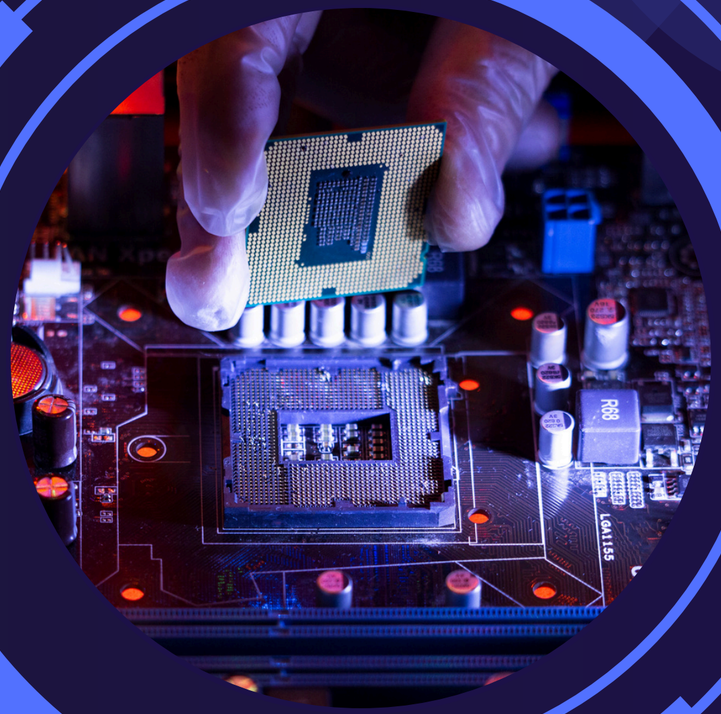
Automated Testing in iOS

How Smart Technologies Are Changing the Way We Work and Innovate.



Gabriela Sanchez Alcaraz

Why do we test?

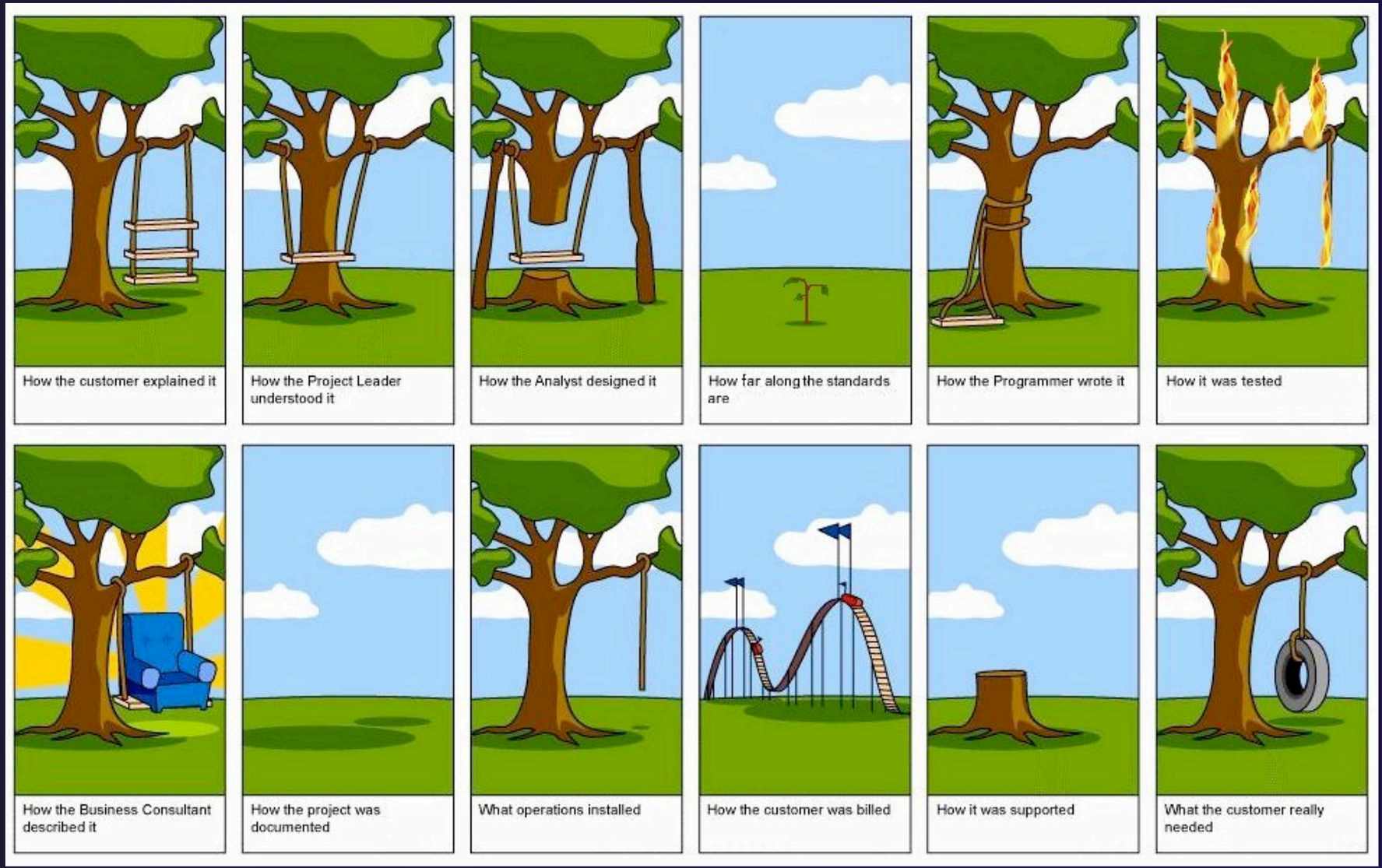
The Problem with Manual Testing

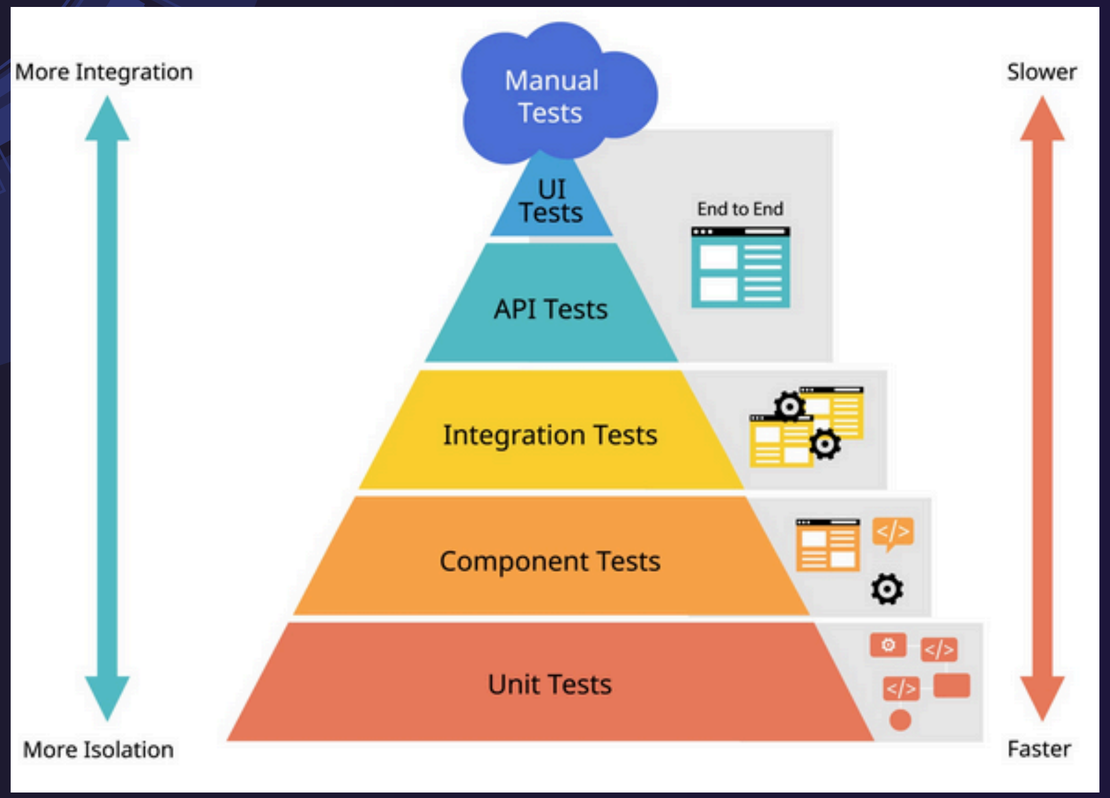
- It is slow and repetitive.
- Humans make mistakes (we forget edge cases).
- It becomes impossible to scale as the app grows.

The Solution: Automated Testing

- what is regression prevention?
- Documentation: Tests document how the code should behave.
- Refactoring Confidence: Allows you to clean up code without fear of breaking logic.
- CI/CD: Essential for continuous integration pipelines.

Why do we test?





The Testing Pyramid

Unit Tests (The Base):

- Tests isolated logic (functions, models).
- Fastest execution.
- Largest quantity (70% of suite).

Integration Tests (The Middle):

- Tests interaction between modules (e.g., ViewModel ↔ Persistence).

UI Tests (End-to-End):

- Simulates user interaction (taps, swipes).
- Slowest execution.
- Smallest quantity (10% of suite).

The Testing

Pyramid

UNIT TEST

- **Task creation logic:** Validating the feature for adding a new task to ensure it incorporates all required details, such as title, description, and deadline
- **User authentication methods**: Ensuring the user login feature correctly validates user credentials and handles errors like incorrect passwords or usernames
- **Notification system**: Checking that the notification system generates the correct alerts for task deadlines or updates

INTEGRATION TEST

- **Task and calendar integration:** Test how new tasks appear in the project calendar, verifying that tasks show up on the correct dates and that updates such as changing a due date are reflected in the calendar.
- **User permissions**: Ensure that changes made by one user (like updating a task status) are correctly reflected for other users with access to the same project, testing the integration between user roles, task management, and real-time updates.

E2E TEST

- **Project lifecycle:** Test the application's ability to handle a project from inception to conclusion by creating a new project, adding team members, creating tasks, updating task status, and archiving or completing the project.
- **Collaboration features:** Check the tool's collaborative capabilities and real-time update features by simulating multiple users working on the same project, including assigning tasks, commenting on tasks, and tracking project progress.

Deep Dive

The Logic & Economics of the Pyramid

01

Open-Box vs. Closed-Box:

- **Unit (Open-Box):** We know the code structure. We test specific paths.
- **UI (Closed-Box):** We are blind. We test strictly as a user.

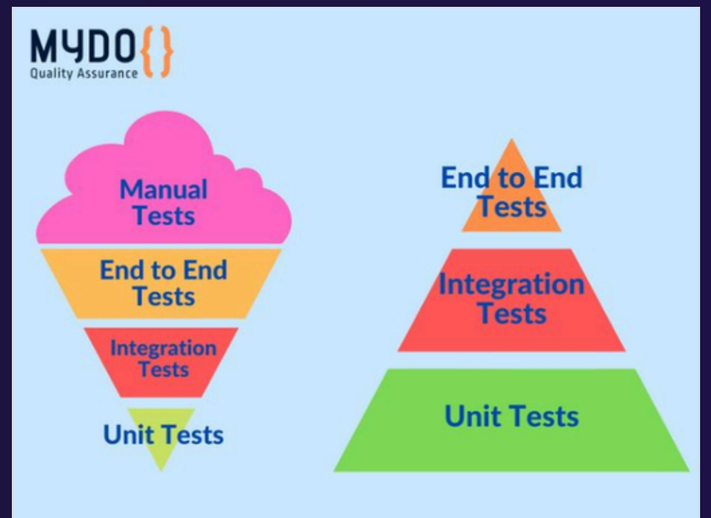
02

The Trade-off:

- **Lower Level:** Faster feedback, cheaper to fix.
- **Higher Level:** More realistic, but brittle and expensive.

Anti-Pattern: The Ice Cream Cone:

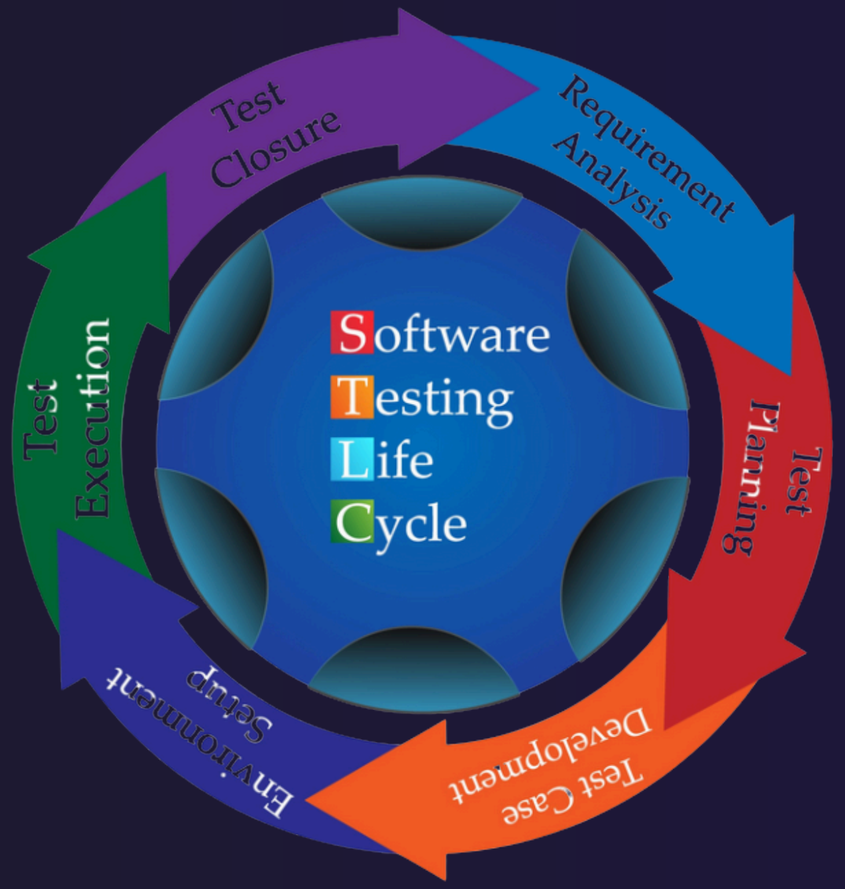
- Having few unit tests and tons of manual/UI tests.
- Result: Slow development cycles and painful debugging.



Concepts popularized by Mike Cohn & Agile methodology.
https://k-hartanto.medium.com/testing-pyramid-and-testing-ice-cream-cone-what-is-the-difference-6ddde3876c20

XCTest & Unit Testing

XCTest: Apple's native testing framework.



Key methods to keep in mind

- **setUpWithError():** Runs before each test. Used to reset state (e.g., clear the database).
- **tearDownWithError():** Runs after each test. Used to clean up memory.

XCTest & Unit Testing

XCTest: Apple's native testing framework.

```swift
import XCTest

class MyTests: XCTestCase {
    func test() {
        let myData = [1, 2, 3]
        let first = myData.first!
        XCTAssert(first < 3)
    }
}
```

```swift
    func testSum() throws {
        XCTAssert(sum(2, 4) == 6)
        XCTAssert(sum(3, 0) == 3)
        XCTAssert(sum(-3, 0) == -3)
    }
```

XCTAssert: The engine of validation.

- XCTAssertTrue(condition)
- XCTAssertEqual(value A, value B)
- XCTAssertNil(value)

GIVEN

WHEN

THEN

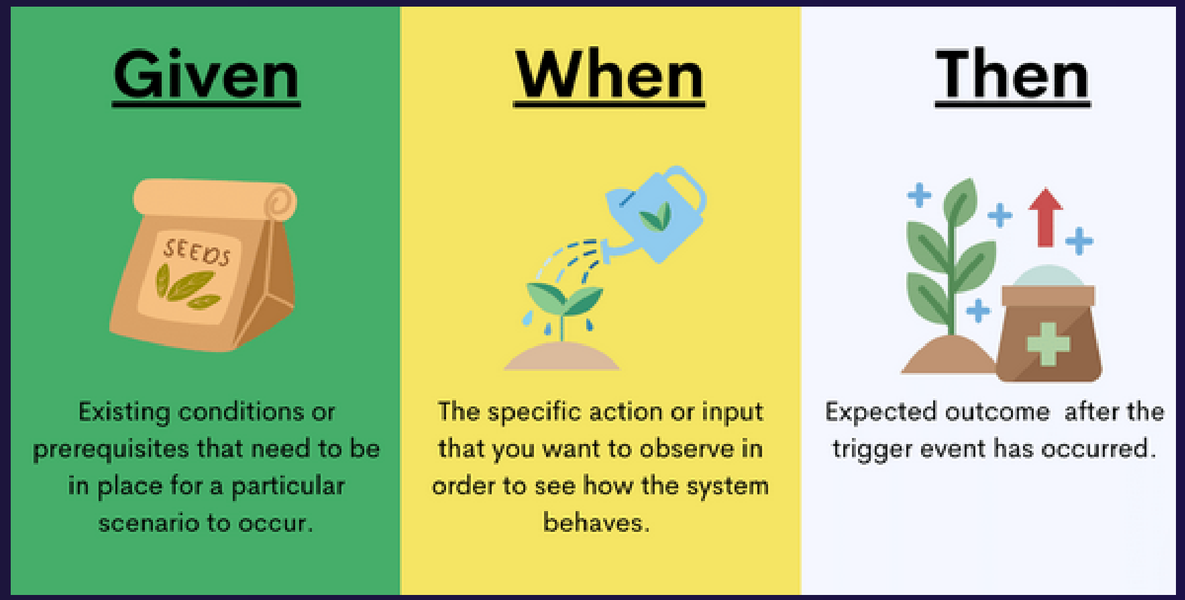
01 Given (The Setup)

This is the state of the world before you touch anything. What data exists? Is the user logged in? Is the cart empty?

02 When (The Action)

This is the one specific thing you are testing. The user taps a button, a function is called, or data loads.

03 Then (The Result)

This is the assertion. Did the world change how we expected? Did the counter go up? Did the error message appear?

```swift
class CartViewModelTests: XCTestCase {

    func test_addItem_incrementsCount() {
        // 1. GIVEN
        let viewModel = CartViewModel()
        viewModel.itemCount = 0

        // 2. WHEN
        viewModel.addItem()

        // 3. THEN
        XCTAssertEqual(viewModel.itemCount, 1, "The item
            count should be 1 after adding an item.")
    }
}
```